

Week 1: The Rise and Decline of Pascal

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

July 12th, 2026

Part One: Language Selection and Context

This paper examines Pascal, a language that dominated computer science education for two decades and then all but vanished from professional practice. Pascal emerged from the turmoil of the late 1960s, when the 1968 NATO conference named the “software crisis” and Dijkstra’s case for structured programming was reframing code as something to be reasoned about rather than patched until it worked. The academic standard, ALGOL 60, was aging, and the committee effort to replace it produced ALGOL 68, a design so sprawling that several members dissented, among them Niklaus Wirth, whose simpler ALGOL W proposal had been passed over. He returned to ETH Zürich and designed Pascal between 1968 and 1970 as a counterstatement: a small, rigorously structured language that a student could learn in its entirety and a modest computer could compile efficiently (Wirth, 1971, 1993).

Part Two: Original Purpose and Design

Wirth’s stated goals were twofold: to provide a language suitable for teaching programming as a systematic discipline built on a small number of fundamental concepts, and to permit implementations that were reliable and efficient on the machines of the day (Wirth, 1971). The target domain was the classroom first and small-scale software second: not an industrial systems language, but the language in which one learns to think about programs. The design features that define Pascal’s identity all serve those goals. Strong static typing, with user-defined enumerations, subranges, records, and sets, let the compiler catch whole categories of student error before a program ever ran. Control flow was strictly structured through begin/end blocks, while, repeat, and case. The language was deliberately small enough to compile in a single pass, which kept compilers fast, simple, and easy to port. Portability through

P-code, in which the compiler emitted instructions for a simple virtual machine, carried Pascal onto microcomputers as UCSD Pascal; by the mid-1980s it was the language of Borland's popular Turbo Pascal and of the AP Computer Science curriculum. Readable, keyword-based syntax completed an identity built on discipline, safety, and teachability (Wirth, 1993).

Part Three: Analysis of Decline

The same decisions that made Pascal an ideal teaching language undermined it as a working one. The most consequential flaw was that an array's size was part of its type: a routine declared to accept an array of ten characters could not accept one of twenty, which made it effectively impossible to write general-purpose libraries—including something as basic as string handling—in standard Pascal (Kernighan, 1981). The standard also lacked modules and separate compilation, so large programs could not be split into independently developed parts.

Kernighan's (1981) widely circulated verdict was that standard Pascal was simply unsuitable for serious programming—and the critique landed just as C became available everywhere. Because the omissions were real, every vendor patched them incompatibly: UCSD, Turbo, and VAX Pascal became mutually incompatible dialects, and real-world Pascal was portable in name only.

The ISO standard's conformant-array fix arrived in 1982, too late and too unevenly implemented to matter, and Wirth himself treated Pascal as a stepping stone, moving on to Modula-2 and then Oberon (Wirth, 1993).

External forces amplified every internal weakness. C spread through universities essentially free with Unix and later gcc, while Pascal's strongest implementations were commercial products tied to one vendor; when Borland faltered in the 1990s, much of the ecosystem went with it. The object-oriented wave then carried industry to C++ and Java, and although Object Pascal and

Delphi were technically capable, they lacked the ecosystem, openness, and standardization to compete. The final blow landed at home: in 1999 the College Board replaced Pascal with C++ (and later Java) in AP Computer Science, evicting the language from the one domain it had unquestionably owned.

Part Four: Historical Connections

Pascal's trajectory matches a recurring trend in language design: the small, elegant academic language loses to the pragmatic industrial one. Empirical adoption research supports what this episode suggests—developers choose languages primarily for libraries, ecosystems, and platform reach, and intrinsic language quality is a secondary factor (Meyerovich & Rabkin, 2013). C had Unix; Pascal had a pedagogy. A second trend: a language that cannot evolve within its standard fragments into dialects, and fragmentation is usually fatal—the official language stays too weak to use while the useful versions stop being one language. Standardization that lags practice does not prevent this decline; it merely documents it.

The broader pattern is that the death of a language is not the death of its ideas. Pascal's convictions—strong static typing, structured control flow, readable declarations—won so completely that they now look unremarkable, living on in Ada, Modula-2, Java, C#, and Go. Python's later capture of the teaching niche shows how the rules changed: a modern teaching language survives by also being an industrial one, with the libraries and job market Pascal never secured. Pascal's lesson for evaluating new languages is that readability and clean design are necessary but not sufficient: long-term viability depends just as heavily on evolvability, ecosystem, and a domain that will still exist in twenty years.

References

Kernighan, B. W. (1981). Why Pascal is not my favorite programming language (Computing Science Technical Report No. 100). Bell Laboratories.

Meyerovich, L. A., & Rabkin, A. S. (2013). Empirical analysis of programming language adoption. In Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications (pp. 1–18). ACM.

<https://doi.org/10.1145/2509136.2509515>

Wirth, N. (1971). The programming language Pascal. *Acta Informatica*, 1(1), 35–63.

<https://doi.org/10.1007/BF00264291>

Wirth, N. (1993). Recollections about the development of Pascal. In T. J. Bergin & R. G. Gibson (Eds.), *History of programming languages—II* (pp. 97–120). ACM.

<https://doi.org/10.1145/234286.1057812>

AI Usage Acknowledgment

Portions of this assignment were completed with assistance from Anthropic’s Claude (via Claude Code). The tool was used to help research the topic, organize my draft paper against the rubric, including verification of source citations.

Anthropic. (2026). Claude [Large language model]. <https://claude.ai>